1. Problem Statement

In traditional Continuous Integration/Continuous Deployment (CI/CD) setups, the deployment mechanism relies on a **push-based model**. This creates several critical challenges:

- **Security Vulnerabilities:** External CI servers (like Jenkins or GitHub Actions) require administrative credentials to access the production Kubernetes cluster, creating a high-value target for attackers.
- **Configuration Drift:** If a system administrator manually alters the live cluster via the command line (e.g., kubectl edit), the live environment no longer matches the configuration stored in the repository. The system state becomes unpredictable.
- **Rollback Complexity:** Reverting a failed deployment often requires re-running complex pipeline scripts rather than simply reverting to a known good state.

2. The Solution (Project Overview)

This project implements a **GitOps-based Continuous Deployment** framework using a **pull-based model**. By deploying an in-cluster controller (ArgoCD or Flux), the Kubernetes cluster actively monitors a centralized Git repository containing declarative configuration files (manifests).

Git becomes the single source of truth. If the live cluster deviates from the Git repository, the GitOps controller automatically detects the drift and forcefully synchronizes the cluster back to the state defined in Git.

3. Project Objectives

- **Establish a Single Source of Truth:** Ensure that 100% of the infrastructure and application configuration is version-controlled in Git.
- **Automate Reconciliation:** Eliminate manual deployments by implementing a controller that automatically pulls changes into the cluster.
- **Enforce Zero-Drift:** Automatically detect and overwrite any manual, unapproved changes made directly to the cluster infrastructure.
- **Improve Security Posture:** Remove the need for external tools to possess cluster credentials by keeping the deployment mechanism entirely inside the cluster firewall.

4. Basic Flow

The pipeline is divided into two distinct phases: CI (Push) and CD (Pull).

1. **Code Commit:** A developer pushes updated application code to the **Source Code Repository**.

2. **Continuous Integration (CI):** A CI tool runs unit tests, builds a new Docker image, and pushes it to an Image Registry (e.g., Docker Hub).
3. **Manifest Update:** The CI tool automatically updates the image tag in the deployment manifests stored in a separate **Configuration Repository**.
4. **Drift Detection:** The GitOps controller (ArgoCD/Flux), operating inside Kubernetes, polls the Configuration Repository and notices the new image tag.
5. **Synchronization (CD):** The controller pulls the updated manifests and applies them to the cluster, spinning up the new pods and tearing down the old ones.

5. Architecture

- **Source Git Repo:** Holds the raw application code (Java, Python, Node.js, etc.).
- **CI Server:** Handles the heavy lifting of building binaries and container images.
- **Container Registry:** Stores the immutable, versioned Docker images.
- **Config Git Repo:** Holds the declarative Kubernetes YAML files, Helm charts, or Kustomize overlays.
- **GitOps Controller (ArgoCD/Flux):** Resides inside the Kubernetes cluster. It has read-only access to the Config Repo and administrative access to the local cluster, acting as the bridge that ensures the cluster perfectly mirrors the Git repository.

